

.NET 10 App Dev Hands-On Lab

MVC Lab 6 –Controllers and Views

This lab walks you through creating the BaseCrudController and finishing the CarsController. Once the controllers are completed, the application's views are added and/or updated. Before starting this lab, you must have completed MVC Lab 5.

Note: Adjust any directory separators to your operating system (e.g. \ for Windows, / for Mac/Linux).

Part 1: Add the Lab's BaseCrudController and Views

Copilot Agent Mode

Prompt:

All work is to be done in the AutoLot.Mvc project.
Add the following Controller and Views

Controller Name: BaseCrudController
Location: Controllers/Base

Empty Razor View: RazorSyntax
Location: Views/Home

Empty Razor View: SimpleService
Location: Views/Home

Empty Razor View Name: Car.cshtml
Location: Views/Cars/DisplayTemplates

Empty Razor View Name: CarWithColors.cshtml
Location: Views/Cars/DisplayTemplates

Empty Razor View Name: Car.cshtml
Location: Views/Cars/EditorTemplates

Empty Razor View Name: _CarList.cshtml
Location: Views/Cars/Partials

Empty Razor View Name: Create
Location: Views/Cars

Empty Razor View Name: Delete
Location: Views/Cars

Empty Razor View Name: Details
Location: Views/Cars

Empty Razor View Name: Edit
Location: Views/Cars

Empty Razor View Name: Index
Location: Views/Cars

Empty Razor View Name: ByMake
Location: Views/Cars

Add the following to the GlobalUsings.cs file:
global using AutoLot.Dal.Repos.Base;
global using AutoLot.Dal.Repos.Interfaces.Base;
global using AutoLot.Models.Entities.Base;
global using Microsoft.AspNetCore.Mvc.Rendering;
global using AutoLot.Mvc.Controllers.Base;

Manual

- Add a new empty Razor View named RazorSyntax to the Views\Home folder
- Add a new empty Razor View named SimpleService to the Views\Home folder.
- Create a new folder named Base in the Controllers folder, and in this folder, create a new class named BaseCrudController.
- Under the Views\Cars folder, create three new folders: DisplayTemplates, EditorTemplates, and Partials.
- Add a new empty Razor view named Car.cshtml under the Views\Cars\DisplayTemplates folder.
- Add a new empty Razor view named CarWithColors.cshtml under the Views\Cars\DisplayTemplates folder.
- Add a new empty Razor view named Car.cshtml under the Views\Cars\EditorTemplates folder.
- Add a new empty Razor view named _CarList.cshtml under the Views\Cars\Partials folder.
- Add the following to the GlobalUsings.cs file:

```
global using AutoLot.Dal.Repos.Base;  
global using AutoLot.Dal.Repos.Interfaces.Base;  
global using AutoLot.Models.Entities.Base;  
global using AutoLot.Mvc.Controllers.Base;  
global using Microsoft.AspNetCore.Mvc.Rendering;
```

Part 2: Update the BaseCrudController

- Update the BaseCrudController to the following:

```
namespace AutoLot.Mvc.Controllers.Base;
```

```
[Route("[controller]/[action]")]
public abstract class BaseCrudController<TEntity>(
    IAppLogger appLogger,
    IBaseRepo<TEntity> baseRepo) : Controller where TEntity : BaseEntity, new()
{
    protected readonly IAppLogger AppLoggerInstance = appLogger;
    protected readonly IBaseRepo<TEntity> BaseRepoInstance = baseRepo;
    protected virtual SelectList GetLookupValues() => null;
    protected TEntity GetOneEntity(int? id) => id == null ? null : BaseRepoInstance.Find(id.Value);
    [Route("/[controller]")]
    [Route("/[controller]/[action]")]
    [HttpGet]
    public virtual IActionResult Index()
        => View(BaseRepoInstance.GetAllIgnoreQueryFiltersAsList());
    [HttpGet("{id?}")]
    public virtual IActionResult Details(int? id)
    {
        if (!id.HasValue)
        {
            return BadRequest();
        }
        var entity = GetOneEntity(id);    return entity == null ? NotFound() : View(entity);
    }
    [HttpGet]
    public virtual IActionResult Create()
    {
        ViewData["LookupValues"] = GetLookupValues();
        return View();
    }
    [HttpPost][ValidateAntiForgeryToken]
    public virtual IActionResult Create(TEntity entity)
    {
        if (!ModelState.IsValid)
        {
            ViewData["LookupValues"] = GetLookupValues();
            return View(entity);
        }
        BaseRepoInstance.Add(entity);
        return RedirectToAction(nameof(Details), new { id = entity.Id });
    }
}
```

```

[HttpGet("{id?}")]
public virtual IActionResult Edit(int? id)
{
    if (!id.HasValue)
    {
        return BadRequest();
    }
    var entity = GetOneEntity(id);
    if (entity == null)
    {
        return NotFound();
    }
    ViewData["LookupValues"] = GetLookupValues();
    return View(entity);
}
[HttpPost("{id}")] [ValidateAntiForgeryToken]
public virtual IActionResult Edit(int id, TEntity entity)
{
    if (id != entity.Id)
    {
        return BadRequest();
    }
    if (!ModelState.IsValid)
    {
        ViewData["LookupValues"] = GetLookupValues();
        return View(entity);
    }
    BaseRepoInstance.Update(entity);
    return RedirectToAction(nameof(Details), new { id = entity.Id });
}
[HttpGet("{id?}")]
public virtual IActionResult Delete(int? id)
{
    if (!id.HasValue)
    {
        return BadRequest();
    }
    var entity = GetOneEntity(id);
    return entity == null ? NotFound() : View(entity);
}
[HttpPost("{id}")] [ValidateAntiForgeryToken]
public virtual IActionResult Delete(int id, TEntity entity)
{
    if (id != entity.Id)
    {
        return BadRequest();
    }
    BaseRepoInstance.Delete(entity);
    return RedirectToAction(nameof(Index));
}
}

```

Part 3: Update the Cars Controller

- Update the CarsController to leverage the new base class and add the two new Action Methods. Update the code to match the following (deleting anything not listed below):

```
namespace AutoLot.Mvc.Controllers;

public class CarsController(IAppLogger appLogger, ICarRepo carRepo, IMakeRepo makeRepo)
    : BaseCrudController<Car>(appLogger, carRepo)
{
    protected override SelectList GetLookupValues()
        => new SelectList(makeRepo.GetAllAsList().OrderBy(m => m.Name), nameof(Make.Id),
        nameof(Make.Name));
    [HttpGet("{makeId}/{makeName}")]
    public IActionResult ByMake(int makeId, string makeName)
    {
        ViewBag.MakeName = makeName;
        return View(carRepo.GetAllByAsList(makeId));
    }
    public IActionResult BadEndPoint() => new OkObjectResult(5);
}
```

- You can hit this with any verb (e.g. using Bruno). When done testing, either comment out this code or add the HttpGet verb:

```
[HttpGet]
public IActionResult BadEndPoint() => new OkObjectResult(5);
```

Part 4: General Views

Step 1: Add the SimpleService View

- Update the SimpleService.cshtml to the following:

```
@model string
<h1>@Model</h1>
```

- Add the following to the _Menu.cshtml partial view:

```
<li class="nav-item dropdown">
    <a class="nav-link dropdown-toggle text-dark" data-bs-toggle="dropdown">
        DI <i class="fa-solid fa-syringe"></i>
    </a>
    <div class="dropdown-menu">
        <a class="dropdown-item text-dark" asp-controller="Home" asp-action="GetServiceOne">
            Service One</a>
        <a class="dropdown-item text-dark" asp-controller="Home" asp-action="GetServiceTwo">
            Service Two</a>
    </div>
</li>
```

Step 2: The RazorSyntax Action Method and View

- Add a new action method named RazorSyntax in the HomeController:

```
[HttpGet]
public IActionResult RazorSyntax([FromServices] ICarRepo carRepo)
{
    var car = carRepo.Find(1);
    return View(car);
}
```

- Update the _Menu.cshtml partial view to include a menu item for the new view:

```
<li class="nav-item">
    <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="RazorSyntax">Razor
Syntax <i class="fa-solid fa-cut"></i></a>
</li>
```

- Update the RazorSyntax view to the following:

```
@model Car
@{
    ViewData["Title"] = "Razor Syntax";
}

<h1>Razor Syntax</h1>

@for (int i = 0; i < 15; i++)
{
    //do something
}
@{
    //Code Block
    var foo = "Foo";
    var bar = "Bar";
    var htmlString = "<ul><li>one</li><li>two</li></ul>";
}
@foo<br />
@htmlString<br />
@foo.@bar<br />
@foo.ToUpper()<br />
@Html.Raw(htmlString)
<hr />
@{
    @:Straight Text
    <div>Value:@Model.Id</div>
    <text>
        Lines without HTML tag
    </text>
    <br />
}

<hr />
@*
    Multiline Comments
    Hi.
```

```
*@
Email Address Handling:<br/>
foo@foo.com = foo@foo.com<br/>
@@foo<br/>
test@foo = test@foo<br/>
test@(foo) = testFoo<br/>
<hr/>
@functions {
    public static IList<string> SortList(IList<string> strings) {
        var list = from s in strings orderby s select s;
        return list.ToList();
    }
}
-----
@{
    var myList = new List<string> {"C", "A", "Z", "F"};
    var sortedList = SortList(myList); //MyFunctions.SortList(myList)
}
@foreach (string s in sortedList)
{
    @s
    @:&nbsp;
}
<hr/>
@{
    Func<dynamic, object> b = @<strong>@item</strong>;
}
This will be bold: @b("Foo")<hr/>
<a asp-controller="Cars" asp-action="Details" asp-route-id="@Model.Id">@Model.PetName</a>
```

Step 3: Update the Index View

- Update the Home/Index.cshtml view to use the DealerInfo passed in from the action method and inject the Keyed Service. Update the view to the following:

```
@model AutoLot.Services.ViewModels.DealerInfo
@inject IServiceProvider serviceProvider
@{
    ViewData["Title"] = "Home Page";
    var service = serviceProvider.GetKeyedService<ISimpleService>(nameof(SimpleServiceOne));
}
<div class="text-center">
    <h1 class="display-4">Welcome to @Model.DealerName</h1>
    <p class="lead">Located in @Model.City, @Model.State</p>
</div>
<div>
    @if (service != null)
    {
        <p>@service.SayHello()</p>
    }
</div>
```

Part 5: The Car Views

Step 1: The Partial and Template Views

- Update the Car.cshtml view under the Views\Cars\DisplayTemplates folder to the following:

```
@model Car
<dl class="row">
  <dt class="col-sm-2">@Html.DisplayNameFor(model => model.MakeId)</dt>
  <dd class="col-sm-10">@Html.DisplayFor(model => model.MakeNavigation.Name)</dd>
  <dt class="col-sm-2">@Html.DisplayNameFor(model => model.Color)</dt>
  <dd class="col-sm-10">@Html.DisplayFor(model => model.Color)</dd>
  <dt class="col-sm-2">@Html.DisplayNameFor(model => model.PetName)</dt>
  <dd class="col-sm-10">@Html.DisplayFor(model => model.PetName)</dd>
  <dt class="col-sm-2">@Html.DisplayNameFor(model => model.Price)</dt>
  <dd class="col-sm-10">@Html.DisplayFor(model => model.Price)</dd>
  <dt class="col-sm-2">@Html.DisplayNameFor(model => model.DateBuilt)</dt>
  <dd class="col-sm-10">@Html.DisplayFor(model => model.DateBuilt)</dd>
  <dt class="col-sm-2">@Html.DisplayNameFor(model => model.IsDrivable)</dt>
  <dd class="col-sm-10">@Html.DisplayFor(model => model.IsDrivable)</dd>
</dl>
```

- Update the CarWithColors.cshtml view under the Views\Cars\DisplayTemplates folder to the following:

```
@model Car
<hr />
<dl class="row">
  <dt class="col-sm-2">@Html.DisplayNameFor(model => model.MakeId)</dt>
  <dd class="col-sm-10">@Html.DisplayFor(model => model.MakeNavigation.Name)</dd>
  <dt class="col-sm-2">@Html.DisplayNameFor(model => model.Color)</dt>
  <dd class="col-sm-10" style="color:@Model.Color">@Html.DisplayFor(model => model.Color)</dd>
  <dt class="col-sm-2">@Html.DisplayNameFor(model => model.PetName)</dt>
  <dd class="col-sm-10">@Html.DisplayFor(model => model.PetName)</dd>
  <dt class="col-sm-2">@Html.DisplayNameFor(model => model.Price)</dt>
  <dd class="col-sm-10">@Html.DisplayFor(model => model.Price)</dd>
  <dt class="col-sm-2">@Html.DisplayNameFor(model => model.DateBuilt)</dt>
  <dd class="col-sm-10">@Html.DisplayFor(model => model.DateBuilt)</dd>
  <dt class="col-sm-2">@Html.DisplayNameFor(model => model.IsDrivable)</dt>
  <dd class="col-sm-10">@Html.DisplayFor(model => model.IsDrivable)</dd>
</dl>
```

- Update the Car.cshtml view under the Views\Cars\EditorTemplates folder to the following:

```
@model Car
<div asp-validation-summary="All" class="text-danger"></div>
<div>
  <label asp-for="MakeId" class="col-form-label"></label>
  <select asp-for="MakeId" class="form-control" asp-items="@ViewBag.LookupValues"></select>
  <span asp-validation-for="MakeId" class="text-danger"></span>
</div>
<div>
  <label asp-for="Color" class="col-form-label"></label>
  <input asp-for="Color" class="form-control"/>
  <span asp-validation-for="Color" class="text-danger"></span>
</div>
<div>
  <label asp-for="PetName" class="col-form-label"></label>
  <input asp-for="PetName" class="form-control" />
  <span asp-validation-for="PetName" class="text-danger"></span>
</div>
<div>
  <label asp-for="Price" class="col-form-label"></label>
  <input asp-for="Price" class="form-control"/>
  <span asp-validation-for="Price" class="text-danger"></span>
</div>
<div>
  <label asp-for="DateBuilt" class="col-form-label"></label>
  <input asp-for="DateBuilt" class="form-control"/>
  <span asp-validation-for="DateBuilt" class="text-danger"></span>
</div>
<div>
  <label asp-for="IsDrivable" class="col-form-label"></label>
  <input asp-for="IsDrivable" />
  <span asp-validation-for="IsDrivable" class="text-danger"></span>
</div>
```

- Update the _CarList.cshtml view under the Views\Cars\Partials folder to the following:

```
@model IEnumerable<Car>
@{
    var showMake = true;
    if (bool.TryParse(ViewBag.ByMake?.ToString(), out bool byMake))
    {
        showMake = !byMake;
    }
}
<p><item-create></item-create></p>
<table class="table">
    <thead>
        <tr>
            @if (showMake)
            {
                <th>@Html.DisplayNameFor(model => model.MakeId) </th>
            }
            <th>@Html.DisplayNameFor(model => model.Color)</th>
            <th>@Html.DisplayNameFor(model => model.PetName)</th>
            <th>@Html.DisplayNameFor(model => model.Price)</th>
            <th>@Html.DisplayNameFor(model => model.DateBuilt)</th>
            <th>@Html.DisplayNameFor(model => model.IsDrivable)</th>
            <th></th>
        </tr>
    </thead>
    <tbody>
        @foreach (var item in Model)
        {
            <tr>
                @if (showMake)
                {
                    <td>@Html.DisplayFor(modelItem => item.MakeNavigation.Name)</td>
                }
                <td>@Html.DisplayFor(modelItem => item.Color)</td>
                <td>@Html.DisplayFor(modelItem => item.PetName)</td>
                <td>@Html.DisplayFor(modelItem => item.Price)</td>
                <td>@Html.DisplayFor(modelItem => item.DateBuilt)</td>
                <td>@Html.DisplayFor(modelItem => item.IsDrivable)</td>
                <td>
                    <item-edit item-id="@item.Id"></item-edit> |
                    <item-details item-id="@item.Id"></item-details> |
                    <item-delete item-id="@item.Id"></item-delete>
                </td>
            </tr>
        }
    </tbody>
</table>
```

Step 2: Update the RazorSyntax View

- Update the bottom of the RazorSyntax view to match the following:

```
This will be bold: @b("Foo")
<hr />
@* If the templates were in the shared folder, you wouldn't need the full path listed*
@Html.DisplayForModel("../Cars/DisplayTemplates/Car.cshtml")
@Html.DisplayForModel("../Cars/DisplayTemplates/CarWithColors.cshtml")
<hr/>
@Html.EditorForModel("../Cars/EditorTemplates/Car.cshtml")
<hr/>
<a asp-controller="Cars" asp-action="Details" asp-route-id="@Model.Id">@Model.PetName</a>
```

Step 3: Update the Car views

- Update the Index.cshtml view in the Views\Cars folder to the following:

```
@model IEnumerable<Car>
@{
    ViewData["Title"] = "Index";
}
<h1>Vehicle Inventory</h1>
<partial name="Partials/_CarList" model="@Model"/>
```

- Update the ByMake.cshtml view in the Views\Cars folder to the following:

```
@model IEnumerable<Car>
@{
    ViewData["Title"] = "Index";
}
<h1>Vehicle Inventory for @ViewBag.MakeName</h1>
@{
    var mode = new ViewDataDictionary(ViewData) {{"ByMake", true}};
}
<partial name="Partials/_CarList" model="Model" view-data="@mode"/>
```

- Update the Details.cshtml view in the Views\Cars folder to the following:

```
@model Car
@{
    ViewData["Title"] = "Details";
}
<h1>Details for @Model.PetName</h1>
@Html.DisplayForModel()
<hr/>
@Html.DisplayForModel("CarWithColors")
<div>
    <item-edit item-id="@Model.Id"></item-edit>
    <item-delete item-id="@Model.Id"></item-delete>
    <item-list></item-list>
</div>
```

- Update the `Create.cshtml` view in the `Views\Cars` folder to the following:

[illegible]

- Update the Edit.cshtml view in the Views\Cars folder to the following:

```
@model Car
@{
    ViewData["Title"] = "Edit";
}
<h1>Edit @Model.PetName</h1>
<hr />
<form asp-area="" asp-controller="Cars" asp-action="Edit" asp-route-id="@Model.Id">
    <div class="row">
        <div class="col-md-4">@Html.EditorForModel()</div>
    </div>
    <div class="d-flex flex-row mt-3">
        <input type="hidden" asp-for="Id" />
        <input type="hidden" asp-for="TimeStamp" />
        <button type="submit" class="btn btn-primary">Save <i class="fa-solid fa-save"></i></button>&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&nbsp;&~
        <item-list></item-list>
    </div>
</form>
@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

- Update the Delete.cshtml view in the Views\Cars folder to the following:

[illegible]

Summary

In this lab, you created the `BaseCrudController` and finished the `Cars Controller`. Then the lab created and or updated the views for the main application.

Next steps

The next lab creates custom validation attributes.